



# PRUEBA TÉCNICA SENIOR: DESARROLLO ASISTIDO POR IA - MVP "BOVITRANS"

## 1. INTRODUCCIÓN Y PROPÓSITO DE LA PRUEBA

Bienvenido al proceso de evaluación técnica para el rol de **Desarrollador Senior**. El objetivo de esta prueba es medir tu capacidad para resolver problemas de software complejos, estructurar arquitecturas escalables, diseñar interfaces prolijas y, de manera primordial, demostrar un nivel sobresaliente de **Desarrollo Asistido por IA (Prompt Engineering Avanzado)**.

A diferencia de las pruebas tradicionales con requerimientos masticados, en este desafío recibirás únicamente una descripción conceptual y de negocio del software que necesitamos construir. Tu primera gran tarea técnica será actuar como Product Owner y Arquitecto, utilizando **Claude** para desglosar esta visión en épicas, historias de usuario, criterios de aceptación y tareas de desarrollo concretas, antes de proceder a la fase de codificación.

## 2. PERFIL DEL CANDIDATO Y STACK TECNOLÓGICO

Esta prueba está calibrada para ingenieros con más de 5 años de experiencia y un fuerte criterio de ingeniería.

| Tecnología / Skill | Nivel Requerido   | Descripción del Uso en la Prueba                                                                |
|--------------------|-------------------|-------------------------------------------------------------------------------------------------|
| HTML5 / CSS3 / JS  | Avanzado          | Interfaz interactiva, sumamente prolija, limpia y con excelente UX/UI.                          |
| Next.js            | Básico/Intermedio | Estructuración del proyecto (App Router recomendado) y manejo de rutas/API.                     |
| SQL y Modelado     | Avanzado          | Base de datos relacional sólida con restricciones lógicas, llaves y consistencia.               |
| API REST           | Avanzado          | Endpoints optimizados, manejo de errores, códigos de estado HTTP correctos.                     |
| Docker             | Básico            | Contenedorización e independencia de servicios (App y Base de Datos).                           |
| GitHub             | Avanzado          | Flujo de Git impecable (Conventional Commits, ramas ordenadas, Pull Requests).                  |
| Prompt Engineering | Avanzado          | Creación de <i>Claude Skills</i> y uso estratégico de IA para análisis, código y documentación. |

### 3. DESCRIPCIÓN DEL PROYECTO (MVP - BOVITRANS)

**BoviTrans** es una plataforma logística diseñada para digitalizar y optimizar el transporte terrestre de ganado vacuno. En el sector ganadero, la coordinación de traslados suele ser caótica y propensa a errores de cálculo en costos y capacidades. Este software busca unificar a los operadores logísticos con los clientes que necesitan mover animales de un punto geográfico a otro, garantizando que los viajes sean seguros, eficientes y financieramente viables.

El núcleo de la aplicación de cara al operador logístico es un **Panel Principal (Dashboard)** interactivo. En esta vista central deben converger todas las solicitudes de transporte entrantes. Cada tarjeta o registro de solicitud debe detallar la información del solicitante, la cantidad exacta de cabezas de ganado a mover, y los puntos geográficos de origen y destino. Para facilitar la toma de decisiones, la interfaz debe integrar un mapa que trace visualmente la ruta, calcule los kilómetros totales de distancia y proyecte el costo total de combustible del viaje de manera dinámica en función del camión que se pretenda asignar.

De manera paralela, el sistema requiere de un **Módulo de Administración de Flotas**. Aquí, el operador registrará y gestionará los camiones disponibles de la empresa. Cada vehículo posee características críticas e inalterables: su patente/matricula, su capacidad máxima de carga (cuántas vacas puede transportar de forma segura en un solo viaje) y su coeficiente de consumo de combustible (medido en litros consumidos por cada kilómetro recorrido,  $\text{L/Km}$ ).

El desafío lógico del MVP radica en la **intersección de ambos módulos**. Cuando el operador asigna un camión a una solicitud de transporte en el Panel Principal, el sistema debe calcular el costo de combustible utilizando la relación matemática:

$$\text{Costo Combustible} = \text{Distancia (Km)} \times \text{Consumo del Vehículo (L/Km)} \times \text{Precio de Combustible por Litro}$$

*(Asume un costo de combustible fijo o parametrizable).* Además, la aplicación debe alertar inmediatamente si la cantidad de ganado solicitada excede la capacidad física del camión asignado, sugiriendo de forma elegante si se requiere realizar múltiples viajes o cambiar de vehículo.

## 4. INSTRUCCIONES DEL DESAFÍO (PASO A PASO)

### FASE 1: Ingeniería de Requerimientos con Claude (Obligatorio)

Antes de escribir código o diseñar la base de datos, debes sentarte con **Claude** y realizar el análisis funcional del proyecto basándote en la descripción del punto anterior.

1. **Configura tu entorno de IA:** Crea un archivo `.claude/skills.json` o `.claude/custom_instructions.md` en tu repositorio que le enseñe a Claude a actuar como tu Analista de Negocios y Arquitecto de Software para BoviTrans.
2. **Generación de Requerimientos:** Solicita a Claude que, a partir de la descripción de BoviTrans, diseñe:
  - Las **Épicas** del proyecto.
  - Las **Historias de Usuario (US)** detalladas (con formato *Como [rol], quiero [acción], para [beneficio]*).
  - Los **Criterios de Aceptación** detallados para cada US (formato *Dado que... Cuando... Entonces...*).
  - Las **Tareas Técnicas (Tasks)** y subtareas de desarrollo específicas para cada historia de usuario.
3. **Entregable de la Fase 1:** Guarda todo este desglose en un archivo llamado `BACKLOG.md` en la raíz de tu proyecto. **Debes incluir en este archivo los prompts o el árbol de conversación que utilizaste con Claude para llegar a este resultado.**

### FASE 2: Diseño de Base de Datos y Dockerización

1. Basándote en el backlog generado, diseña el modelo de datos SQL.
2. Configura un archivo `docker-compose.yml` que separe el contenedor de la aplicación (Next.js) del contenedor de la base de datos (PostgreSQL/MySQL).
3. Asegura que la base de datos se inicialice automáticamente con tablas y datos semilla mediante un archivo `init.sql`.

### FASE 3: Desarrollo de la Aplicación

1. Implementa el backend (API REST en Next.js) y el frontend siguiendo las historias de usuario que definiste en la Fase 1.
2. Integra mapas interactivos (Leaflet, Mapbox, u OpenStreetMap) para el trazado de rutas.
3. Asegura una UI/UX impecable, prolija, moderna y responsiva.

### FASE 4: Documentación Asistida

Utiliza a Claude para generar un archivo `DOCUMENTACION.md` sumamente riguroso que explique la arquitectura de la solución, las decisiones de diseño del modelo de datos, la API y cómo correr localmente el proyecto a través de Docker.

## 5. RÚBRICA DE EVALUACIÓN (SENIOR)

| Criterio de Evaluación              | Peso | Qué se medirá                                                                                                                                                              |
|-------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingeniería de Requerimientos con IA | 25%  | La calidad del BACKLOG.md, la profundidad de las historias de usuario y tareas propuestas, el diseño de tu archivo .claude/skills.json y el valor agregado de tus prompts. |
| Modelado de Datos SQL               | 20%  | Estructura de tablas, relaciones lógicas, uso correcto de llaves primarias/foráneas, indexación y consistencia de datos.                                                   |
| Arquitectura de Software y API      | 20%  | Estructura limpia de carpetas en Next.js, endpoints REST consistentes, manejo de estados HTTP, control de errores y modularidad del código.                                |
| UI/UX y Trazado de Mapas            | 20%  | Prolijidad visual, limpieza en la presentación de datos logísticos, interactividad fluida del mapa y manejo interactivo de advertencias de carga.                          |
| Dockerización e Infraestructura     | 15%  | Levantar la aplicación completa con un único comando (docker-compose up --build), persistencia en volúmenes y variables de entorno correctas.                              |

## 6. ENTREGA Y PLAZOS

- **Plazo de Entrega:** Tienes exactamente **8 días** corridos para completar el desafío a partir de la fecha de recepción de esta pauta.
- **Formato de Entrega:**
  1. Repositorio de GitHub publico (brindar url para el acceso a los revisores asignados).
  2. El desarrollo debe realizarse en una rama de características (ej. feature/bovitrans-mvp) y se debe abrir un **Pull Request (PR)** hacia la rama main.
  3. El PR debe contar con una descripción rica donde expliques tus decisiones arquitectónicas y cómo te apoyaste en Claude para optimizar tus tiempos de entrega.